PAGE 1: INTRODUCTION TO OOP

What is a Programming Paradigm?

A **Programming Paradigm** is a fundamental style or "way" of programming. It dictates how a programmer structures and organizes code to solve a problem. Examples include Procedural, Object-Oriented, Functional, and Logical programming paradigms.

Procedural Programming vs. Object-Oriented Programming

Feature	Procedural Programming (POP)	Object-Oriented Programming (OOP)
Approach	Top-down approach.	Bottom-up approach.
Focus	Focuses on functions (procedures) and actions.	Focuses on objects and data.
Data Security	Data is accessible globally; less secure.	Data is hidden (encapsulated); highly secure.
Extensibility	Difficult to add new data or functions.	Easy to add new data/functions via inheritance.

Features of OOP

- **Classes & Objects:** The basic building blocks.
- **Encapsulation:** Wrapping data and methods into a single unit.
- **Abstraction:** Hiding internal implementation details from the user.
- **Inheritance:** Reusing existing code by deriving new classes from existing ones.
- **Polymorphism:** The ability of a variable, function, or object to take on multiple forms.

Advantages of OOP

- **Reusability:** "Write once, use multiple times" through inheritance.
- **Security:** Data hiding and encapsulation protect sensitive data.
- **Maintainability:** Modular structure makes it easier to troubleshoot, modify, and maintain.
- **Scalability:** Easy to scale up the codebase for large, complex projects.

Applications of OOP

OOP is extensively used in developing Client-Server Systems, Object-Oriented Databases, Real-time System Design, Simulation and Modeling systems, Artificial Intelligence, and developing GUI applications.

Popular OOP Languages

- **Java:** Platform-independent, heavily enforces OOP principles.
- **C++:** Adds OOP features to the procedural C language.
- **Python:** Multi-paradigm, uses dynamic typing and elegant OOP syntax.
- **C#:** Developed by Microsoft, heavily used in enterprise and game development (.NET/Unity).

PAGE 2: CLASS AND OBJECT

Class

A **Class** is a user-defined blueprint or prototype from which objects are created. It represents the set of properties or methods that are common to all objects of one type. It is a logical entity (does not consume memory when declared).

Object

An **Object** is a real-world entity and a basic unit of OOP. It is an instance of a class. When a class is defined, no memory is allocated until an object of that class is instantiated.

Characteristics of Objects

- **State:** Represented by the attributes or properties (variables) of an object.
- **Behavior:** Represented by the methods (functions) of an object.
- **Identity:** A unique name or ID given to an object to distinguish it from other objects.

Creating Objects

To use a class, you must create an object (instance) of it. Syntax varies by language (e.g., `Car myCar = new Car();` in Java/C#).

Constructors

A **Constructor** is a special member function invoked automatically when an object is created. It initializes the object's state. It shares the exact same name as the class.

- **Default Constructor:** A constructor that takes no arguments. It assigns default values to object attributes.
- **Parameterized Constructor:** A constructor that takes arguments, allowing different objects to be initialized with different values at the time of creation.
- **Copy Constructor:** A constructor that initializes an object using another object of the same class (e.g., `Car car2 = new Car(car1);`).

Destructor

A **Destructor** is a special member function called automatically when an object's lifetime ends or is explicitly deleted. It is used to free up resources (memory, network sockets, file pointers) allocated to the object. In C++, it is denoted by a tilde (e.g., `~Car()`).

this Pointer / this Keyword

The **this** keyword (or pointer in C++) is a reference to the current instance of the class on which a method is being invoked. It is used to resolve naming conflicts between class attributes and local parameters, and to pass the current object as a parameter to other methods.

PAGE 3: FOUR PILLARS OF OOP

1. Encapsulation

Definition: The mechanism of wrapping data (variables) and code (methods) together as a single unit (like a capsule). It also restricts direct access to some of the object's components.

Advantages: Prevents accidental data modification, increases security, and allows making fields read-only or write-only.

Example: A `BankAccount` class where the `balance` variable is private, and can only be modified through `deposit()` and `withdraw()` methods.

2. Abstraction

Definition: The concept of hiding complex internal implementation details and showing only the essential, relevant features to the user.

Advantages: Reduces system complexity, improves code readability, and isolates changes in the backend from the front-end user.

Example: Driving a car. You press the accelerator to speed up. You do not need to know how the fuel injection and internal combustion engine work to drive it.

3. Inheritance

Definition: The process by which one class (child/subclass) acquires the properties and functionalities of another class (parent/superclass).

Advantages: Promotes code reusability, establishes an "IS-A" relationship, and is essential for implementing runtime polymorphism.

Example: A `Dog` class inherits from an `Animal` class. The `Dog` class automatically has properties like `age` and `eat()`, and can add its own like `bark()`.

4. Polymorphism

Definition: The ability of a single entity (function, operator, or object) to take on multiple forms and behave differently depending on the context.

Advantages: Allows programmers to use a single interface for a general class of actions, making code cleaner and highly scalable.

Example: A `draw()` function. If called on a `Circle` object, it draws a circle. If called on a `Square` object, it draws a square. Same function name, different behavior.

PAGE 4: INHERITANCE

Definition

Inheritance is a core mechanism in OOP that allows a new class (derived/child class) to inherit the data members and member functions of an existing class (base/parent class). It models an "IS-A" relationship (e.g., A Car IS-A Vehicle).

Types of Inheritance

- **Single Inheritance:** A subclass inherits from exactly one parent class. (A → B)
- **Multiple Inheritance:** A subclass inherits from more than one parent class simultaneously. (A, B → C). Note: Java and C# do not support multiple inheritance with classes to avoid ambiguity, but C++ does.
- **Multilevel Inheritance:** A subclass inherits from a parent class, which in turn inherits from another parent class. (A → B → C)
- **Hierarchical Inheritance:** Multiple subclasses inherit from a single parent class. (A → B, A → C, A → D)
- **Hybrid Inheritance:** A combination of two or more types of inheritance. It often leads to complex relationships.

Method Overriding

Method overriding occurs when a subclass provides a specific implementation for a method that is already defined in its parent class. The method in the subclass must have the same name, return type, and parameters as the one in the parent class. It allows a class to inherit from a superclass but modify behavior as needed.

Virtual Functions

A **Virtual Function** is a member function in the base class that you expect to redefine in derived classes. In C++, using the `virtual` keyword tells the compiler to perform dynamic linkage or late binding on the function. It ensures that the correct overridden function is called for an object, regardless of the type of reference/pointer used for the call.

Dynamic Binding

Also known as **Late Binding**. It means that the code associated with a given procedure call is not known until the time of the call at runtime. When virtual functions are used, the compiler delays the binding of the function call to the actual code until runtime based on the actual object type, facilitating runtime polymorphism.

PAGE 5: POLYMORPHISM

Polymorphism (from Greek "many forms") is categorized into two main types based on when the binding takes place: Compile-Time and Run-Time.

Compile-Time Polymorphism (Static Binding)

This type of polymorphism is resolved during the compilation process. The compiler knows exactly which function to invoke based on the method signature. It is faster but less flexible.

- **Function Overloading:** Defining multiple functions with the exact same name in the same scope, but with different parameters (different number of parameters, different types, or different order). The compiler decides which one to call based on the arguments passed.
- **Operator Overloading:** Providing a special, user-defined meaning to an existing operator (like +, -, *, ==) for user-defined data types (classes). For example, overloading the '+' operator to add two `ComplexNumber` objects together.

Run-Time Polymorphism (Dynamic Binding)

This type of polymorphism is resolved during the execution of the program. It provides high flexibility as the system decides which method to execute on the fly.

- **Method Overriding:** As discussed, a child class provides a different implementation of a method defined in its parent class.
- **Role of Virtual Functions:** In C++, Run-Time polymorphism is achieved through base class pointers pointing to derived class objects, utilizing `virtual` functions.

Static Binding vs. Dynamic Binding

Feature	Static Binding (Early Binding)	Dynamic Binding (Late Binding)
Resolution Time	Compile-time.	Run-time.
Mechanism	Function and Operator Overloading.	Method Overriding (Virtual functions).
Execution Speed	Faster, as everything is decided before execution.	Slower, as resolution occurs during execution.
Flexibility	Less flexible.	Highly flexible and dynamic.

PAGE 6: ABSTRACTION & ENCAPSULATION

Data Hiding

Data hiding is an OOP concept that restricts direct access to internal data members of a class. It is the core implementation of encapsulation, ensuring that object state can only be altered through designated interfaces (methods).

Access Specifiers

Access specifiers (or modifiers) define the visibility and accessibility of a class's members (variables and functions).

- **Public:** Members are accessible from outside the class, anywhere in the program.
- **Private:** Members are accessible *only* from within the class itself. They are hidden from the outside world.
- **Protected:** Members are accessible within the class and to any subclasses (derived classes) that inherit from it, but not to the outside world.

Friend Function and Friend Class (Specific to C++)

- **Friend Function:** A function that is not a member of a class but is granted access to its private and protected members. It is declared inside the class using the `friend` keyword.
- **Friend Class:** A class that is granted access to the private and protected members of another class. Used when two classes are closely related.

Abstract Class

An **Abstract Class** is a class that cannot be instantiated (you cannot create objects of it). It acts as a strict blueprint for derived classes. It contains at least one abstract method (or pure virtual function). Subclasses must provide implementations for these abstract methods.

Pure Virtual Function

In C++, a **Pure Virtual Function** is a virtual function that has no implementation in the base class and is declared by assigning it to zero (e.g., `virtual void draw() = 0;`). A class containing at least one pure virtual function becomes an abstract class.

Interface

An **Interface** is a pure abstract concept. It contains *only* declarations of methods (no implementations) and constant variables. It acts as a contract; any class that implements the interface must define all of its methods. (Java and C# heavily use the `interface` keyword to achieve multiple inheritance of type).

PAGE 7: MEMORY CONCEPTS

Stack Memory

Stack Memory is used for static memory allocation and the execution of threads. It stores primitive variables and references to objects. Memory allocation and deallocation are handled automatically (LIFO structure) as methods are called and returned. It is fast, but limited in size.

Heap Memory

Heap Memory is used for dynamic memory allocation. Objects and large arrays are stored here. Memory must be managed manually (in C/C++) or by a Garbage Collector (in Java/C#). It is larger than the stack but slower to access.

new Operator

The `new` operator is used to allocate memory dynamically on the Heap. It returns the memory address of the allocated block. In OOP, `new` also implicitly calls the constructor of the class to initialize the object.

delete Operator

The `delete` operator (in C++) is used to explicitly deallocate memory that was previously allocated via `new` on the heap. Using `delete` calls the object's destructor before freeing the memory.

Dynamic Memory Allocation

The process of allocating memory space during the execution time (run-time) of a program. It gives developers flexibility to allocate exactly the amount of memory needed based on user input or file sizes, preventing waste.

Garbage Collection

Garbage Collection (GC) is an automatic memory management feature found in modern languages (Java, C#, Python). The GC periodically runs in the background to look for objects on the Heap that are no longer referenced by any part of the program, and automatically frees that memory. It prevents memory leaks without requiring explicit `delete` commands.

Object Lifetime

The lifetime of an object is the time spanning from its creation (constructor invocation) to its destruction (destructor/garbage collection). For stack objects, lifetime ends when the variable goes out of scope. For heap objects, lifetime ends when explicitly deleted or collected by the GC.

PAGE 8: ADVANCED OOP CONCEPTS

Static Members and Functions

- **Static Members (Variables):** A variable shared by all objects of a class. It belongs to the class itself rather than any specific instance. Only one copy exists in memory, regardless of how many objects are created.
- **Static Functions:** A method that can be called directly on the class itself, without creating an object. A static function can only access static variables and other static functions.

Constant Objects and Methods

In C++, a `const` object is one whose state cannot be changed after initialization. Similarly, a `const` member function guarantees that it will not modify any member variables of the object it is called upon.

Inline Functions

An **Inline Function** is a request to the compiler to replace the function call with the actual code of the function during compilation. This reduces the overhead of function calls (pushing/popping to the stack) for small, frequently used functions, speeding up execution time.

Namespaces

A **Namespace** is a declarative region that provides a scope to identifiers (names of types, functions, variables). It is used to organize code into logical groups and prevent name collisions (e.g., having two classes named `Logger` in different libraries).

Templates (Generics)

Templates (C++) or **Generics** (Java/C#) allow you to write generic, type-independent code. You can create a single function or class that can operate on different data types (e.g., an `Array<T>` that can store integers, floats, or Strings without rewriting the class).

Exception Handling (try, catch, throw)

Exception handling is a robust mechanism to handle runtime errors, ensuring the normal flow of the application is maintained.

- **try:** A block of code where exceptions might occur. If an error happens, it stops execution and passes control to the catch block.
- **throw:** Used to explicitly signal that an exceptional condition (error) has occurred. It "throws" an exception object.
- **catch:** A block of code designed to handle the specific exception thrown by the try block, allowing the program to recover gracefully.

PAGE 9: UML & RELATIONSHIPS

UML Class Diagram

The **Unified Modeling Language (UML)** is a standardized visual language used in software engineering. A Class Diagram describes the structure of a system by showing the system's classes, their attributes, operations (methods), and the relationships among objects.

Object Relationships

Objects in a system rarely work in isolation. They collaborate through various types of relationships:

- **Association:** A broad relationship where two classes are connected, but neither owns the other. It implies a "uses-a" relationship. Example: A `Teacher` is associated with a `Student`. They can exist independently.
- **Aggregation:** A specialized form of association representing a "has-a" relationship, but with a weak lifecycle dependency. The child can exist independently of the parent. Example: A `Department` has `Teachers`. If the Department is closed, the Teachers still exist.
- **Composition:** A strict form of aggregation representing a "part-of" relationship with a strong lifecycle dependency. If the parent is destroyed, the child is destroyed. Example: A `House` and a `Room`. If the house is demolished, the rooms cease to exist.
- **Dependency:** A weak relationship indicating that a change in one class may affect another class that uses it. Often, class A depends on class B if it takes class B as a parameter in a method.
- **Generalization:** The UML term for Inheritance. It shows an "IS-A" relationship between a general superclass and a specific subclass. Represented by a solid line with a hollow arrow pointing to the superclass.

PAGE 10: VIVA & INTERVIEW TOPICS

SOLID Principles

Five design principles for writing maintainable OOP code:
S: Single Responsibility (A class should have one job).
O: Open/Closed (Open for extension, closed for modification).
L: Liskov Substitution (Subclasses must be substitutable for base classes).
I: Interface Segregation (Many specific interfaces are better than one general-purpose interface).
D: Dependency Inversion (Depend on abstractions, not concretions).

Coupling and Cohesion

- **Coupling:** The degree of interdependence between classes. *Goal: Low Coupling.*
- **Cohesion:** The degree to which the elements inside a class belong together. *Goal: High Cohesion.*

Shallow Copy vs. Deep Copy

- **Shallow Copy:** Copies the exact memory addresses of pointers. Both objects point to the same dynamic memory. Modifying one affects the other.
- **Deep Copy:** Allocates separate memory for pointers and copies the actual values. The objects are completely independent. Requires a custom Copy Constructor.

Copy Constructor vs Assignment Operator

A **Copy Constructor** initializes a *newly* created object using an existing one (`A a2 = a1;`). An **Assignment Operator** replaces the data of an *already existing* object with another (`a2 = a1;`).

Virtual Destructor

When a derived class object is deleted using a base class pointer, only the base class destructor is called (causing a memory leak in the derived class). A **Virtual Destructor** ensures that the derived class destructor is called first, safely freeing all memory.

Object Slicing

Occurs when a derived class object is assigned to a base class variable (passed by value, not by pointer/reference). The extra attributes unique to the derived class are "sliced off", leaving only the base class portion.

The Diamond Problem

An ambiguity that arises in multiple inheritance when classes B and C inherit from A, and class D inherits from both B and C. If D calls a method from A, the compiler doesn't know whether to use B's path or C's path. Resolved in C++ using `virtual inheritance`.

Real-World Example of OOP

A highly cohesive real-world example: A **Library Management System**. `Person` is an abstract class. `Student` and `Librarian` inherit from `Person`. A `Library` has an **Aggregation** of `Books`. Encapsulation

protects the Book's checkout status. Polymorphism is used via an `issueResource()` method handling both Books and Magazines differently.